Design of the Code

- High level description of approach
  The approach that is used inside the code was to structure the program around small building blocks that are also composable, each responsible for a part of the process which includes parsing, validation, and more. The program will read a specification of a grammar and parse it, validate it, and generate the valid haskell code. The code is written in a way that is modular, where each step is independent but linked through the data flow and composition
- High level structure of code
  The program can be divided into a few main sections including the data types, parsing, validation, generating the code and finally utility functions. Below are the explanations of each part.
    1. Data types: includes ADT, Rule, Symbol, and Modifier. These represent the parsed grammar structure.
    2. Parsing: the process of implementing parsing logic to the input grammar so that it can turn to data structures, this is done using parser combinators.
    3. Validation: performs checks such as duplicate rule detection, undefined symbol references, and left recursion.
    4. Generating the code: converts the ADT into haskell code, producing type and parser definitions
    5. Utility functions: helper combinators (sepBy, sepBy1, string, etc.) and small reusable components such as capitalize
- Code architecture choices
  A data driven approach was used to separate parsing logic from code generation. Rules and symbols are represented as structured data, which allows independent processing, validating and transforming. Because of this, the grammar extension process is much easier as it can be done without rewriting the parsing logic from the beginning. The overall code resembles that of a compiler pipeline, which is scalable

Parsing

- Usage of parser combinators
  Parser combinators are heavily used in this program as it can compose small parsing functions into more powerful ones. Each of the combinators returns a parser value, which can be combined using functional operators such as <|>, <*>, and >>=. By using this approach, the parser is now modular and compositional, meaning adding a new grammar only required defining new small parsers and combining them
- Choices made in creating parsers
  Since we have to parse a BNF grammar, the parser is designed for a typical BNF grammar. To explain more, since there are many rules, all of the rules will correspond to one parser only and that is ruleParser. Furthermore, each symbol can be a terminal, nonterminal, macro, or modified symbol.
- Functor, Applicative, and Monad usage
  Functors are used to transform parsed results without changing the structure of the parser. Applicatives allowed sequencing parsers without manually threading state. Monad was used in multiple places where the next parser depends on the result of the previous one.

Functional programming

- Small modular functions
  Each part of the program performs one task only such as parsing rules, validating them, or generating code. Each part contains multiple functions like symbolToParser or generateConstructor which are all small and testable. This modularity enhances readability and makes debugging straightforward.
- Composing small functions together
  Inside the code, outputs of a function may be used as inputs for another. This can be seen in bnfParser, where it calls many ruleParser, which combines several smaller parsers like productionParser and symbolParser. In one perspective, this type of structure is quite similar to that of grammar definitions, which are hierarchical.
- Declarative style
  The functions inside the code don't describe the parsing steps, rather they express what the structure should look like.

Haskell language features used

In the program, the use of functors, applicatives and monads can be found in a lot of places, these types of typeclasses are used to define composable parsing logic. Not only that, ADT, Rule and Symbol, which are considered as custom types are also used to help model complex structures and ensure that invalid grammar won't occur accidentally. With this, boilerplate will be reduced and reusability will be improved.

Higher order functions and composition

Some places that uses higher order functions are:

1. sepBy which takes two parsers and returns a new parser
2. findLeftRecursions use list comprehensions and recursive calls to express graph traversal in a functional way. Other validation functions also implement higher order functions
3. generateHaskellCode, one of the key functions inside the program, uses concatMap and function composition to transform complex data structures in an elegant way.

Description of extension

The parser is now extended to handle parameterized nonterminals and modifiers as well. These modifiers include *, +, and ?. The parameterized nonterminals allow grammar rules to include parameters which can increase expressiveness. Modifiers like ZeroOrMore, OneOrMore, or Optional will require recursive parsing and special handling in code generation to output [type] or Maybe type structures in haskell. With these extensions, the grammar system can be more realistic to real language parsers.